



CourseCube®
(Formerly Java Learning Center)

Spring Boot 2

Module 1

Data Access with Boot - Part 1

Author

Srinivas Dande





Module 1 : Spring Data Access with Boot - Part 1

Objectives: In this Module, You will learn Data Access with Spring Boot Applications with JDBC, Data Sources and Various Spring Boot features.

Topics:

- ✓ Spring -JDBC Flow
- ✓ Developing Spring -JDBC Example
- ✓ Understanding the Problems with this Example.
- ✓ First Spring Boot Example
- ✓ Spring Boot Features.
- ✓ Starters
- ✓ Auto Configurations
- ✓ Exploring Starters
- ✓ Exploring Auto Configurations
- ✓ Exploring YAML Files
- ✓ Using Hikari DataSource
- ✓ Using Tomcat DataSource
- ✓ Using DBCP2
- ✓ Using DBCP
- ✓ Using JdbcTemplate
- ✓ Using NamedParameterJdbcTemplate
- ✓ Using Logging with Boot
- ✓ Understanding Spring Boot Start-Up Tasks
- ✓ Exploring CommandLineRunner
- ✓ Exploring @SpringBootApplication
- ✓ Exploring @ComponentScan



Spring Boot 2

- 1) Spring offers a powerful way to build web applications, with support for dependency injection, transaction management, polyglot persistence, application security, first-hand REST API support, an MVC framework and a lot more.
- 2) Spring applications have always required significant configuration and, which sometimes build up a lot of complexity during development. That's where Spring Boot comes in.
- 3) The Spring Boot project aims to make building web application with Spring much faster and easier. The guiding principle of Boot is convention over configuration.

Important features in Spring Boot:

1. **Spring Boot Starters**
2. **Auto Configurations**
3. **Spring Boot Initializer**
4. **Spring Boot CLI**
5. **Dev Tools**
6. **Externalized Configurations**
7. **YAML Support**
8. **Embedded Containers**
9. **Actuators**
10. **Spring Boot Admin**



data-access-jdbc-demo1: Spring and JDBC with Maven

data-access-jdbc-demo1: Working Steps:

A) Setup Database

```
create database myjlcdb5;
use myjlcdb5;
create table mycustomers(
cid int primary key,
cname char(10),
email char(10),
phone long,
city char(10)
);
```

B) Create the maven Project.

- 1) Open the Eclipse required Workspace
- 2) Select New -> Maven Project.
- 3) Check the checkbox for create Simple Project (skip the Archetype selection) and Click on Next Button.

- 4) Provide the following

Group Id: **com.jlcindia.spring**
Artifact Id : **data-access-jdbc-demo1**
Version: **1.0**
packaging: **jar**
Name: **data-access-jdbc-demo1**
and Click On finish button.

- 5) Add the maven-compiler-plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.1</version>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>
```

- 6) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

Select the Project, right click and then select Maven -> Update Project.



7) Update pom.xml by adding the following dependencies.

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.2.5.RELEASE</version>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.5.RELEASE</version>
</dependency>

<dependency>
    <groupId>commons-dbcp</groupId>
    <artifactId>commons-dbcp</artifactId>
    <version>1.4</version>
</dependency>

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.6</version>
</dependency>
```

8) Create the Package called **com.jlcindia.spring**

9) Write JLCAppConfig.java (Bean Configuration class)

- Configure DataSource.
- Configure JdbcTemplate

10)Write the following Files

- Customer.java
- CustomerRowMapper.java

11)Write the DAO and its Implementation class

- CustomerDAO.java
- CustomerDAOImpl.java

12)Write Client Program - MyClient and Run it.



Spring -JDBC Example with Maven

data-access-jdbc-demo1: Files required

1. MyClient.java	
2. CustomerDAO.java	
3. CustomerDAOImpl.java	
4. Customer.java	
5. CustomerRowMapper.java	
6. JLCAppConfig.java	
7. pom.xml	

1)MyClient.java

```
package com.jlcindia.spring;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class MyClient {
    public static void main(String[] args) {

        ApplicationContext ctx=new AnnotationConfigApplicationContext(JLCAppConfig.class);
        CustomerDAO custDAO =(CustomerDAO) ctx.getBean("custDAO");

        Customer cust = new Customer(104,"hello","hello@jlc",12345,"Delhi");
        custDAO.addCustomer(cust);

        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");

        List<Customer> mylist2 = custDAO.getCustomersByCity("Blore");
        mylist2.forEach(mycust -> System.out.println(mycust));

        System.out.println("Done!!!");
    }
}
```



2) CustomerDAO.java

```
package com.jlcindia.spring;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {
    public void addCustomer(Customer cust);
    public List<Customer> getAllCustomers();
    public List<Customer> getCustomersByCity(String city);
}
```

3) CustomerDAOImpl.java

```
package com.jlcindia.spring;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository("custDAO")
public class CustomerDAOImpl implements CustomerDAO{

    @Autowired
    JdbcTemplate jtemp;

    @Override
    public void addCustomer(Customer cust) {
        String SQL="insert into mycustomers values(?,?,?,?)";
        jtemp.update(SQL,
            cust.getCid(),cust.getCname(),cust.getEmail(),cust.getPhone(),cust.getCity());
    }

    @Override
    public List<Customer> getAllCustomers() {
        String SQL="select * from mycustomers";
        List<Customer> mylist = jtemp.query(SQL, new CustomerRowMapper());
        return mylist;
    }
}
```



```
@Override
public List<Customer> getCustomersByCity(String city) {
    String SQL="select * from mycustomers where city=?";
    List<Customer> mylist = jtemp.query(SQL, new CustomerRowMapper(),city);
    return mylist;
}

}
```

4)Customer.java

```
package com.jlcindia.spring;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Customer {

    private int cid;
    private String cname;
    private String email;
    private int phone;
    private String city;

    public Customer() {}
    public Customer(String cname, String email, int phone, String city) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }

    public Customer(int cid, String cname, String email, int phone, String city) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }

    //Setters and Getters
    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + email + ", " + phone + ", " + city + "]";
    }
}
```



5)CustomerRowMapper.java

```
package com.jlcindia.spring;

import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.jdbc.core.RowMapper;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class CustomerRowMapper implements RowMapper<Customer> {

    @Override
    public Customer mapRow(ResultSet rs, int rowNum) throws SQLException {
        Customer cust=new Customer();

        cust.setCid(rs.getInt(1));
        cust.setCname(rs.getString(2));
        cust.setEmail(rs.getString(3));
        cust.setPhone(rs.getInt(4));
        cust.setCity(rs.getString(5));

        return cust;
    }
}
```

6)JLCAppConfig.java

```
package com.jlcindia.spring;

import javax.sql.DataSource;

import org.apache.commons.dbcp.BasicDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Configuration
@ComponentScan(basePackages = {"com.jlcindia.spring"})
public class JLCAppConfig {
```



```
@Bean(name = "dataSource")
```

```
public DataSource getDataSource() {
```

```
    BasicDataSource bds = new BasicDataSource();  
    bds.setDriverClassName("com.mysql.jdbc.Driver");  
    bds.setUrl("jdbc:mysql://localhost:3306/myjlcdb5");  
    bds.setUsername("root");  
    bds.setPassword("srinivas");  
    bds.setMaxActive(5);  
    return bds;  
}
```

```
@Bean(name = "jtemp")
```

```
public JdbcTemplate getJdbcTemplate(DataSource dataSource) {  
    JdbcTemplate jtemp = new JdbcTemplate(dataSource);  
    return jtemp;  
}  
}
```

7)pom.xml

```
<project ...>
```

```
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.spring</groupId>  
<artifactId>data-access-jdbc-demo1</artifactId>  
<version>1.0</version>
```

```
<name>data-access-jdbc-demo1</name>  
<description>This is Spring and JDBC example</description>  
<url>https://www.jlcindia.com</url>
```

```
<properties>  
<java.version>1.8</java.version>  
<spring.version>5.2.5.RELEASE</spring.version>  
</properties>
```

```
<dependencies>
```

```
<dependency>  
<groupId>org.springframework</groupId>  
<artifactId>spring-context</artifactId>  
<version>${spring.version}</version>  
</dependency>
```



```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-jdbc</artifactId>
<version>${spring.version}</version>
</dependency>

<dependency>
<groupId>commons-dbcp</groupId>
<artifactId>commons-dbcp</artifactId>
<version>1.4</version>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.1</version>
<configuration>
<source>${java.version}</source>
<target>${java.version}</target>
</configuration>
</plugin>
</plugins>
</build>
</project>
```